

Table of Contents

Introduction.....	2
Problem Description.....	2
Knowledge Representation and Rule Design	2
Inference and Flow Control.....	3
Wider AI Context.....	4
Ethical, Commercial and Social Issues	4
Ambiguity and Misleading Results.....	4
Complexity and Uncertainty	5
Knowledge Acquisition and Inductive Learning	5
Human Factors	6
Evaluation and Testing	6
Methodology.....	6
Functional Path Coverage	7
1. Startup and Rule Initialization	8
2. Dynamic Inference and Mid Interaction	9
3. Final Output and Explanation.....	9
4. Rule Firing Trace	10
5. Robustness and Invalid Input Handling.....	10
Conclusion	11
Reference	12
Appendix	13
Appendix A (CLIPS Code).....	13
Appendix B (Panopto link).....	22
Appendix C (Freshwater Arthropod Identification).....	22

Table of Figures

Figure 1: Inference Flow within the CLIPS Expert System	3
Figure 2: CLIPS environment after loading the rule base.	8
Figure 3: system initialised and awaiting user input.	8
Figure 4: Mid interaction showing user question and response.	9
Figure 5: Example of final identification output message.	9
Figure 6: Rule activation trace during inference process.	10
Figure 7: System handling invalid user input gracefully.	10
Figure 8: Original identification tree	22

Introduction

This report explains how a basic expert system was developed to identify freshwater arthropods using a simple decision tree. The system asks the user a few yes/no questions and gives a result based on the answers, along with a short explanation. The aim was to explore how rule based logic and knowledge representation can be used in a practical and understandable way as part of an AI project.

The tool was built using CLIPS 6.30, which is a forward chaining system that matches rules and facts using the Rete algorithm (Aldrobi et al., 2013). The whole decision tree was turned into a set of rules that trigger automatically based on the users input. This removed the need for writing if/else conditions. All testing was done on macOS using Parallels Desktop to make sure it works well with the Windows version used in the university.

This small project shows how expert systems can be used to support biological classification. It also shows how rule-based systems can follow a clear reasoning path, similar to what a real expert might do when deciding.

Problem Description

This project focuses on identifying freshwater arthropods based on visible features like the number of legs, presence of tails, and wings. In many lab settings, this process is usually done by hand, which can take time and may lead to different results depending on the person doing it, especially if they are not experts. The system aims to make this task quicker and more consistent by turning the experts reasoning into a set of clear, rule based steps. This shows how symbolic AI can be used for simple and practical classification tasks.

Knowledge Representation and Rule Design

The system represents knowledge using a typical rule based approach. It relies on a set of facts and rules that connect physical traits like the number of legs, tail presence, or wing type to specific arthropod types. This structure helps the system follow a similar logic to how a human expert would think during classification.

In clips, the knowledge is organised using three main parts: deftemplates define the data format, facts store the users input, and defrules link conditions to outcomes. As answers are given, new facts are asserted, and the system moves

forward automatically by triggering the next rule. Clips were chosen because it is simple to use and works well for small, portable expert systems. Developed by NASA, it supports clean rule writing and makes it easier to test and improve the system when needed.

Inference and Flow Control

The system uses forward chaining to manage the reasoning process, progressing step by step as the user responds. It dynamically checks which rules apply to the current facts and acts accordingly. Each user response adds new facts to the system, which naturally guides the session toward a final conclusion.

This way of reasoning is close to how a human might solve such a task, starting from simple details and narrowing things down gradually. Thanks to the Rete algorithm, only the relevant rules are evaluated, which improves efficiency. And since the logic is rule based, it is easy to adjust or add new cases without having to change the rest of the system.

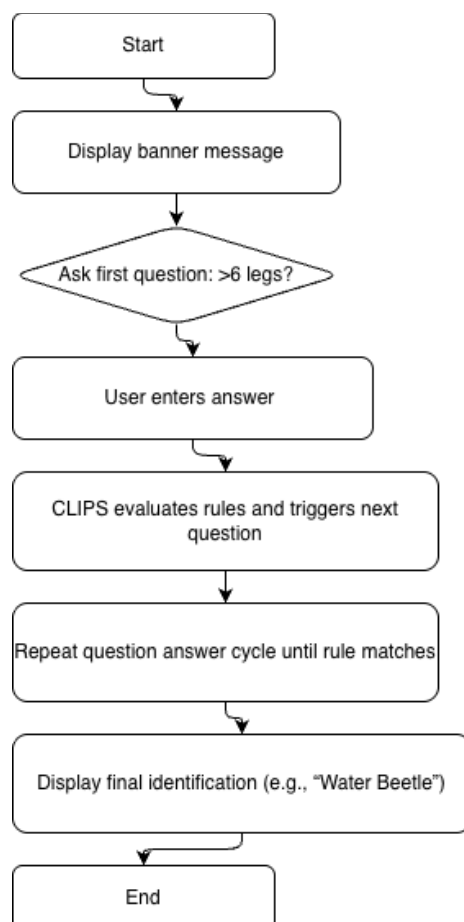


Figure 1: Inference Flow within the CLIPS Expert System

Wider AI Context

This expert system is an example of symbolic artificial intelligence, where knowledge is written as clear rules instead of learned from data like in machine learning. One of the main benefits of symbolic systems is that each step in the reasoning process can be explained. This makes them useful in places like schools or scientific projects, where it is important to understand how a decision was made, not just what the answer is.

Lately, there has been more interest in symbolic AI again, especially because of its role in explainable AI (XAI). Many researchers believe that these systems can work alongside machine learning models to improve how easy it is to trust and understand automated decisions (Bellucci, 2023). The freshwater arthropod expert system built in this project is a small example of this idea. It shows how using structured rules can lead to results that are both clear and easy to check.

Ethical, Commercial and Social Issues

Although this expert system was created for learning purposes, it still brings up some of the usual concerns seen in AI, like fairness, openness, and user confidence. To support trust, the system gives a short reason after each result, helping users understand why it gave that answer, not just what the answer is. This kind of explanation is useful in learning, especially when the focus is on understanding how things work.

From a business view, systems like this could help save time on training by offering steady results, but it is still necessary to check them properly so they do not cause mistakes. On the social side, expert systems can be useful in making technical knowledge easier to understand for people who are not experts. As Kovari (2024) mentioned, building trust in AI means not only being accurate but also being clear something that was kept in mind while designing this tool.

Ambiguity and Misleading Results

Even though the system usually works well, there are still times when the result might be unclear or misleading. Since it depends fully on the answers users give, a wrong or unsure input might send the system in the wrong direction, causing it to give the wrong result. Also, some physical features between species can be very similar, so asking only yes/no questions does not always match real differences.

To avoid this as much as possible, the questions were kept short and clear, and the system ignores any invalid answers to stop mistakes. Still, with this kind of rule based setup, it is hard to remove all confusion. In the future, it might help to add extra questions to double check unclear answers or show a message that explains how sure the system is.

Complexity and Uncertainty

Expert systems using rules work fine when the problem is simple and the choices are clear. But if we add more rules, the system can get harder to control. Sometimes rules mix together in a way that makes the answer confusing or not clear.

Also, uncertainty is a big issue. In real situations, not all info is complete or easy to understand. Some details might be between yes and no, but rule systems need a clear answer. So, the result might be wrong if the input is not clear.

This is a known problem in symbolic AI. Like Weibel (1991) said, adding too many rules makes the system harder to follow, and it does not work well when the cases are close to each other.

Knowledge Acquisition and Inductive Learning

In this project, the knowledge base was created by manually turning an experts decision tree into a list of simple rules the system could use. Each part of the tree like how many legs the organism has, or whether it has a tail was written as a condition the system could check during classification.

While the system does not learn on its own, the rules were improved over time through testing. This helped make sure the results were consistent and that every outcome made sense and could be explained clearly.

The approach was based on the idea that proper knowledge in an expert system should always be both accurate and understandable. As Hossjer et al. (2022) said, knowledge is like a justified belief, meaning that every answer should have a reason behind it. That's why the system always shows a short explanation with each result, and the rules can still be adjusted later if new examples are added.

Human Factors

When building the system, the way people use it was an important point to think about. Since it works by asking yes or no questions, the wording had to be short and clear so users would not get confused. The questions also go in the same order as the decision tree, which makes it easier for users to guess what the next question will be about.

To make things smoother, the system can notice if the user gives a wrong input and asks them to try again. This helps avoid breaking the process and makes it feel more like a real conversation. The interface in clips is very simple, but that actually helps because users can clearly see each step of the decision. Overall, the focus was on making everything easy to understand, with helpful feedback and a clear structure.

Evaluation and Testing

This section checks the system's accuracy, completeness, robustness, and reproducibility.

Test Environment: Each run started from a clean state using:

```
(clear)
(load "C:\\IntelligentSystems\\Arthropod_Project\\arthropod_expert.clp")
(reset)
(run)
```

Methodology

1. Path coverage:
All 13 branches were tested to confirm they lead to valid outcomes.
2. Explanation check:
Each final result was checked to confirm it includes a short explanation (why message) that matches the users previous answers.
3. Robustness:
Incorrect inputs (example: "yess", numbers) were rejected and the user was asked to try again.
4. Reproducibility:
Some runs were repeated with watch rules to confirm rules fired in the same order each time.

Functional Path Coverage

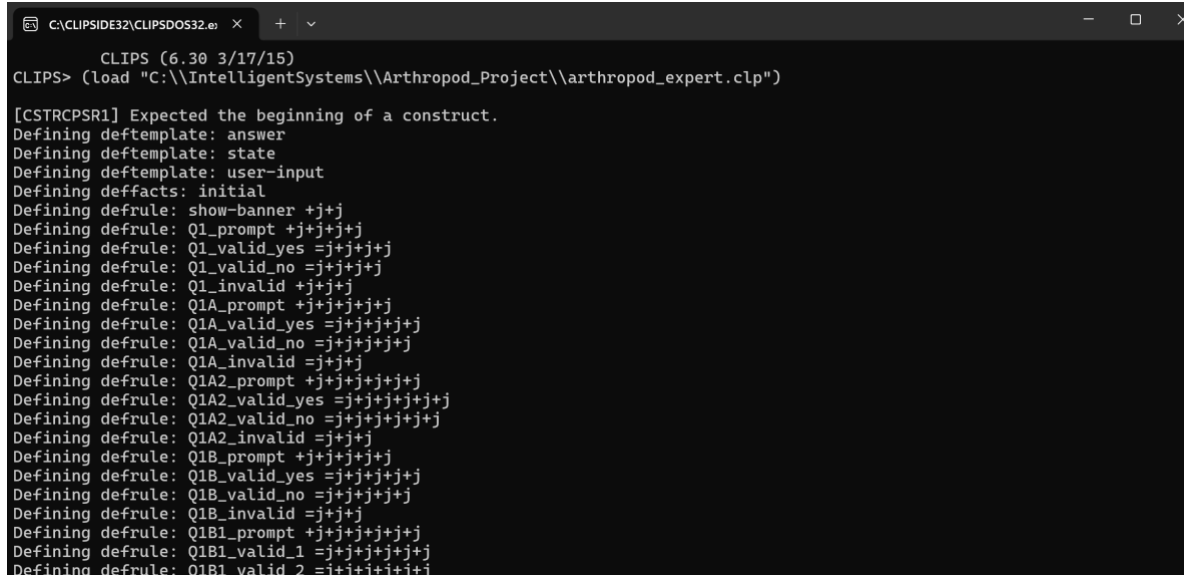
The table below lists the 13 representative cases: inputs , expected identification and the actual outputs.

Table 1: Summary of functional test cases and corresponding identification outcomes.

Case No.	User Input Sequence (Summary)	Expected Identification	Actual Output
1	>6 legs = yes → 8 legs = yes	Water Spider or Mite	Water Spider or Mite
2	>6 legs = yes → 8 legs = no → big claws = yes	Freshwater Crayfish	Freshwater Crayfish
3	>6 legs = yes → 8 legs = no → big claws = no	Freshwater Shrimp or Hog louse	Freshwater Shrimp or Hog louse
4	>6 legs = no → tails = yes → tail count = 1 → feathery = yes	Alderfly Larva	Alderfly Larva
5	>6 legs = no → tails = yes → tail count = 1 → feathery = no	Water Stick Insect or Water Scorpion	Water Stick Insect or Water Scorpion
6	>6 legs = no → tails = yes → tail count = 2 → exactly 2 = yes	Stonefly Nymph	Stonefly Nymph
7	>6 legs = no → tails = yes → tail count = >1 → longer = yes → thin hairy = yes	Mayfly Nymph	Mayfly Nymph
8	>6 legs = no → tails = yes → tail count = >1 → longer = yes → thin hairy = no	Damselfly Nymph	Damselfly Nymph
9	>6 legs = no → tails = yes → tail count = >1 → longer = no	Dragonfly Nymph	Dragonfly Nymph
10	>6 legs = no → tails = no → segmented = no → wings overlap = yes	Water Boatman	Water Boatman
11	>6 legs = no → tails = no → segmented = no → wings overlap = no	Water Beetle	Water Beetle
12	>6 legs = no → tails = no → segmented = yes → stubby = yes	Dragonfly Nymph	Dragonfly Nymph
13	>6 legs = no → tails = no → segmented = yes → stubby = no	Caseless Caddis Fly or Beetle Larvae	Caseless Caddis Fly or Beetle Larvae

1. Startup and Rule Initialization

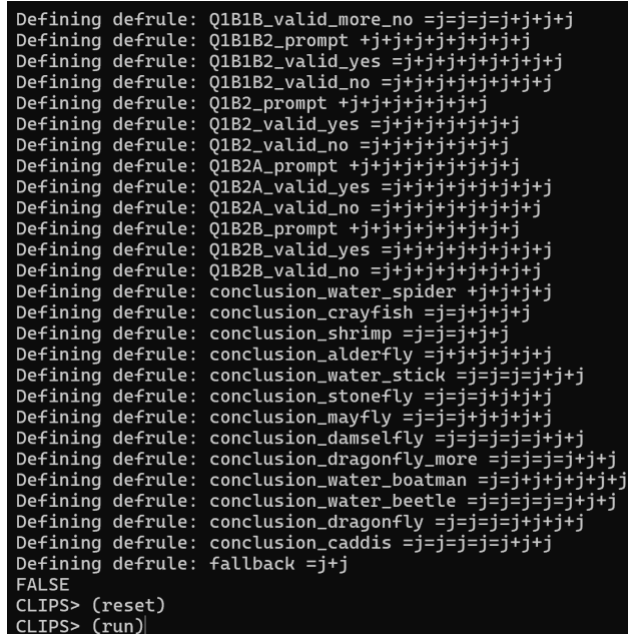
The system started without any errors. All the rules and data were loaded correctly. After a quick reset, it was ready to ask questions.



```
C:\CLIPSIDE32\CLIPSDOS32.0> CLIPS (6.30 3/17/15)
CLIPS> (load "C:\\IntelligentSystems\\Arthropod_Project\\arthropod_expert.clp")

[CSTRCPSR1] Expected the beginning of a construct.
Defining deftemplate: answer
Defining deftemplate: state
Defining deftemplate: user-input
Defining deffacts: initial
Defining defrule: show-banner +j+j
Defining defrule: Q1_prompt +j+j+j+j
Defining defrule: Q1_valid_yes =j+j+j+j+j
Defining defrule: Q1_valid_no =j+j+j+j+j
Defining defrule: Q1_invalid +j+j+j
Defining defrule: Q1A_prompt +j+j+j+j+j+j
Defining defrule: Q1A_valid_yes =j+j+j+j+j+j
Defining defrule: Q1A_valid_no =j+j+j+j+j+j+j
Defining defrule: Q1A_invalid =j+j+j+j
Defining defrule: Q1A2_prompt +j+j+j+j+j+j+j
Defining defrule: Q1A2_valid_yes =j+j+j+j+j+j+j+j
Defining defrule: Q1A2_valid_no =j+j+j+j+j+j+j+j
Defining defrule: Q1A2_invalid =j+j+j+j
Defining defrule: Q1B_prompt +j+j+j+j+j+j
Defining defrule: Q1B_valid_yes =j+j+j+j+j+j+j
Defining defrule: Q1B_valid_no =j+j+j+j+j+j+j+j
Defining defrule: Q1B_invalid =j+j+j+j
Defining defrule: Q1B1_prompt +j+j+j+j+j+j+j+j
Defining defrule: Q1B1_valid_1 =j+j+j+j+j+j+j+j+j
Defining defrule: Q1B1_valid_2 =j+j+j+j+j+j+j+j+j+j
```

Figure 2: CLIPS environment after loading the rule base.



```
Defining defrule: Q1B1B_valid_more_no =j=j=j=j+j+j+j+j
Defining defrule: Q1B1B2_prompt +j+j+j+j+j+j+j+j+j+j
Defining defrule: Q1B1B2_valid_yes =j+j+j+j+j+j+j+j+j+j
Defining defrule: Q1B1B2_valid_no =j+j+j+j+j+j+j+j+j+j
Defining defrule: Q1B2_prompt +j+j+j+j+j+j+j+j+j+j
Defining defrule: Q1B2_valid_yes =j+j+j+j+j+j+j+j+j+j
Defining defrule: Q1B2_valid_no =j+j+j+j+j+j+j+j+j+j
Defining defrule: Q1B2A_prompt +j+j+j+j+j+j+j+j+j+j
Defining defrule: Q1B2A_valid_yes =j+j+j+j+j+j+j+j+j+j+j
Defining defrule: Q1B2A_valid_no =j+j+j+j+j+j+j+j+j+j+j
Defining defrule: Q1B2B_prompt +j+j+j+j+j+j+j+j+j+j+j
Defining defrule: Q1B2B_valid_yes =j+j+j+j+j+j+j+j+j+j+j
Defining defrule: Q1B2B_valid_no =j+j+j+j+j+j+j+j+j+j+j
Defining defrule: conclusion_water_spider +j+j+j+j+j
Defining defrule: conclusion_crayfish =j=j=j+j+j+j+j
Defining defrule: conclusion_shrimp =j=j=j+j+j+j+j
Defining defrule: conclusion_alderfly =j+j+j+j+j+j+j+j
Defining defrule: conclusion_water_stick =j=j=j=j=j+j+j+j
Defining defrule: conclusion_stonefly =j=j=j+j+j+j+j+j
Defining defrule: conclusion_mayfly =j=j=j+j+j+j+j+j+j
Defining defrule: conclusion_damselfly =j=j=j=j=j+j+j+j+j
Defining defrule: conclusion_dragonfly_more =j=j=j=j+j+j+j+j
Defining defrule: conclusion_water_boatman =j=j+j+j+j+j+j+j+j
Defining defrule: conclusion_water_beetle =j=j=j=j=j+j+j+j+j
Defining defrule: conclusion_dragonfly =j=j=j=j+j+j+j+j+j
Defining defrule: conclusion_caddis =j=j=j=j=j+j+j+j+j+j
Defining defrule: fallback =j+j
FALSE
CLIPS> (reset)
CLIPS> (run)
```

Figure 3: system initialised and awaiting user input.

2. Dynamic Inference and Mid Interaction

After starting, the system asks a question. Based on the users answers, it keeps asking the next one automatically. This shows how the system updates its logic step by step.

```
=== Freshwater Arthropod Identification ===  
Answer questions to identify freshwater arthropods  
  
1. Does the arthropod have more than 6 legs? (yes/no): no  
2. Are tails present? (yes/no): yes  
3. How many tails? (1/2/>1): >1  
4. Are tails longer than body width? (yes/no):
```

Figure 4: Mid interaction showing user question and response.

3. Final Output and Explanation

Once enough answers are given, the system shows the final result. It also gives a short reason to explain how that answer was chosen.

```
=== Freshwater Arthropod Identification ===  
Answer questions to identify freshwater arthropods  
  
1. Does the arthropod have more than 6 legs? (yes/no): yes  
2. Does it have 8 legs (4 pairs)? (yes/no): no  
3. Does it have big claws? (yes/no): yes  
  
==> Identification: Freshwater Crayfish  
Why: big claws
```

Figure 5: Example of final identification output message.

4. Rule Firing Trace

To see how the system thinks step by step, the command watch rules was used. It showed which rules were activated one by one during the session, ending with the final decision.

```
CLIPS> (reset)
CLIPS> (watch rules)
CLIPS> (run)
FIRE    1 show-banner: f-1

=== Freshwater Arthropod Identification ===
Answer questions to identify freshwater arthropods

FIRE    2 Q1_prompt: f-4,*,f-2
1. Does the arthropod have more than 6 legs? (yes/no): yes
FIRE    3 Q1_valid_yes: f-4,f-5,*
FIRE    4 Q1A_prompt: f-7,f-6,*,f-2
2. Does it have 8 legs (4 pairs)? (yes/no): no
FIRE    5 Q1A_valid_no: f-7,f-8,f-6,*
FIRE    6 Q1A2_prompt: f-10,f-6,f-9,*,f-2
3. Does it have big claws? (yes/no): yes
FIRE    7 Q1A2_valid_yes: f-10,f-11,f-6,f-9,*
FIRE    8 conclusion_crayfish: f-13,f-6,f-9,f-12

==> Identification: Freshwater Crayfish
Why: big claws

[PRCCODE4] Execution halted during the actions of defrule conclusion_crayfish.
```

Figure 6: Rule activation trace during inference process.

5. Robustness and Invalid Input Handling

The system was tested with wrong inputs like “yess” instead of “yes”. It rejected the invalid answer, asked the user to try again, and continued without crashing. This shows that input checking works properly.

```
=== Freshwater Arthropod Identification ===
Answer questions to identify freshwater arthropods

1. Does the arthropod have more than 6 legs? (yes/no): yess
Please answer yes or no.
```

Figure 7: System handing invalid user input gracefully.

Conclusion

This project demonstrated how a rule-based expert system can support the identification of freshwater arthropods by following a structured decision making process. The decision tree was successfully converted into a series of logical rules, allowing the system to ask relevant questions and provide clear explanations for its conclusions. The use of CLIPS ensured that the reasoning process was transparent, consistent, and easy to verify.

Testing across all thirteen classification paths showed that the system performed reliably and handled invalid inputs without errors. While the rule based approach may struggle with uncertain or overlapping traits, it remains effective for well-defined tasks. Future improvements could include more species, confidence indicators, or a simple visual interface to enhance usability.

Reference:

Aldrobi, M., Anastasiadis, S., Khalife, B., Kontogianis, K. and Demori, R. (2013) *CLIPS enhanced with objects, backward chaining and explanation facilities*. Available at: <https://ntrs.nasa.gov/citations/19960002929> (Accessed: 1 November 2025).

Bellucci, M. (2023) 'Symbolic approaches for explainable artificial intelligence'. *Hal.science*. Available at: <https://theses.hal.science/tel-04469103/> (Accessed: 4 November 2025).

Hössjer, O., Díaz-Pachón, D.A. and Rao, J.S. (2022) 'A Formal Framework for Knowledge Acquisition: Going beyond Machine Learning'. *Entropy* 24(10), p. 1469. doi: <https://doi.org/10.3390/e24101469>.

Kovari, A. (2024) 'AI for Decision Support: Balancing Accuracy, Transparency, and Trust Across Sectors'. *Information* 15(11), pp. 725–725. Available at: <https://www.mdpi.com/2078-2489/15/11/725>.

Weibel, R. (1991) *Amplified intelligence and rule-based systems*. Available at: https://www.researchgate.net/publication/292543147_Amplified_intelligence_and_rule-based_systems.

Appendix

Appendix A (CLIPS Code)

```
:: Freshwater Arthropod Identification

(clear)

(deftemplate answer
  (slot question)
  (slot value))

(deftemplate state
  (slot current-question))

(deftemplate user-input
  (slot value)
  (slot valid))

(defacts initial
  (state (current-question start))
  (user-input (valid no)))

;; ===== BANNER =====
(defrule show-banner
  ?s <- (state (current-question start))
  =>
  (printout t crlf "=== Freshwater Arthropod Identification ===" crlf)
  (printout t "Answer questions to identify freshwater arthropods" crlf crlf)
  (assert (answer (question banner) (value shown)))
  (modify ?s (current-question Q1)))

;; ===== Q1: >6 LEGS? =====
(defrule Q1_prompt
  (state (current-question Q1))
  (not (answer (question Q1)))
  (user-input (valid no))
  =>
  (printout t "1. Does the arthropod have more than 6 legs? (yes/no): ")
  (bind ?response (lowercase (read)))
  (assert (user-input (value ?response) (valid yes))))

(defrule Q1_valid_yes
  ?s <- (state (current-question Q1))
  ?i <- (user-input (value yes) (valid yes))
  (not (answer (question Q1)))
  =>
  (assert (answer (question Q1) (value yes)))
  (modify ?s (current-question Q1A))
  (retract ?i)
  (assert (user-input (valid no))))

(defrule Q1_valid_no
  ?s <- (state (current-question Q1))
  ?i <- (user-input (value no) (valid yes))
  (not (answer (question Q1)))
  =>
  (assert (answer (question Q1) (value no)))
  (modify ?s (current-question Q1B))
  (retract ?i)
  (assert (user-input (valid no))))

(defrule Q1_invalid
  ?i <- (user-input (value ?v&:(not (or (eq ?v yes) (eq ?v no)))) (valid yes))
  (state (current-question Q1))
  =>
  (printout t "Please answer yes or no." crlf)
  (retract ?i)
  (assert (user-input (valid no))))

;; ===== Q1A: 8 LEGS? =====
(defrule Q1A_prompt
  (state (current-question Q1A))
  (answer (question Q1) (value yes))
  (not (answer (question Q1A)))
  (user-input (valid no))
  =>
  (printout t "2. Does it have 8 legs (4 pairs)? (yes/no): ")
```

```

(bind ?response (lowercase (read)))
(assert (user-input (value ?response) (valid yes))))

(defrule Q1A_valid_yes
?s <- (state (current-question Q1A))
?i <- (user-input (value yes) (valid yes))
(answer (question Q1) (value yes))
(not (answer (question Q1A)))
=>
(assert (answer (question Q1A) (value yes)))
(modify ?s (current-question conclusion))
(retract ?i)
(assert (user-input (valid no))))

(defrule Q1A_valid_no
?s <- (state (current-question Q1A))
?i <- (user-input (value no) (valid yes))
(answer (question Q1) (value yes))
(not (answer (question Q1A)))
=>
(assert (answer (question Q1A) (value no)))
(modify ?s (current-question Q1A2))
(retract ?i)
(assert (user-input (valid no))))

(defrule Q1A_invalid
?i <- (user-input (value ?v&:(not (or (eq ?v yes) (eq ?v no)))) (valid yes))
(state (current-question Q1A))
=>
(printout t "Please answer yes or no." crlf)
(retract ?i)
(assert (user-input (valid no))))

;; ===== Q1A2: BIG CLAWS? =====
(defrule Q1A2_prompt
(state (current-question Q1A2))
(answer (question Q1) (value yes))
(answer (question Q1A) (value no))
(not (answer (question Q1A2)))
(user-input (valid no))
=>
(printout t "3. Does it have big claws? (yes/no): ")
(bind ?response (lowercase (read)))
(assert (user-input (value ?response) (valid yes))))

(defrule Q1A2_valid_yes
?s <- (state (current-question Q1A2))
?i <- (user-input (value yes) (valid yes))
(answer (question Q1) (value yes))
(answer (question Q1A) (value no))
(not (answer (question Q1A2)))
=>
(assert (answer (question Q1A2) (value yes)))
(modify ?s (current-question conclusion))
(retract ?i)
(assert (user-input (valid no))))

(defrule Q1A2_valid_no
?s <- (state (current-question Q1A2))
?i <- (user-input (value no) (valid yes))
(answer (question Q1) (value yes))
(answer (question Q1A) (value no))
(not (answer (question Q1A2)))
=>
(assert (answer (question Q1A2) (value no)))
(modify ?s (current-question conclusion))
(retract ?i)
(assert (user-input (valid no))))

(defrule Q1A2_invalid
?i <- (user-input (value ?v&:(not (or (eq ?v yes) (eq ?v no)))) (valid yes))
(state (current-question Q1A2))
=>
(printout t "Please answer yes or no." crlf)
(retract ?i)
(assert (user-input (valid no))))

;; ===== Q1B: TAILS PRESENT? =====
(defrule Q1B_prompt
(state (current-question Q1B))

```

```

(answer (question Q1) (value no))
(not (answer (question Q1B)))
(user-input (valid no))
=>
(printout t "2. Are tails present? (yes/no): ")
(bind ?response (lowercase (read)))
(assert (user-input (value ?response) (valid yes))))

(defrule Q1B_valid_yes
?s <- (state (current-question Q1B))
?i <- (user-input (value yes) (valid yes))
(answer (question Q1) (value no))
(not (answer (question Q1B)))
=>
(assert (answer (question Q1B) (value yes)))
(modify ?s (current-question Q1B1))
(retract ?i)
(assert (user-input (valid no))))

(defrule Q1B_valid_no
?s <- (state (current-question Q1B))
?i <- (user-input (value no) (valid yes))
(answer (question Q1) (value no))
(not (answer (question Q1B)))
=>
(assert (answer (question Q1B) (value no)))
(assert (answer (question tail-count) (value 0)))
(modify ?s (current-question Q1B2))
(retract ?i)
(assert (user-input (valid no))))

(defrule Q1B_invalid
?i <- (user-input (value ?v&:(not (or (eq ?v yes) (eq ?v no)))) (valid yes))
(state (current-question Q1B))
=>
(printout t "Please answer yes or no." crlf)
(retract ?i)
(assert (user-input (valid no))))

;; ===== Q1B1: TAIL COUNT =====
(defrule Q1B1_prompt
(state (current-question Q1B1))
(answer (question Q1) (value no))
(answer (question Q1B) (value yes))
(not (answer (question tail-count)))
(user-input (valid no))
=>
(printout t "3. How many tails? (1/2/>1): ")
(bind ?response (read))
(assert (user-input (value ?response) (valid yes))))

(defrule Q1B1_valid_1
?s <- (state (current-question Q1B1))
?i <- (user-input (value 1) (valid yes))
(answer (question Q1) (value no))
(answer (question Q1B) (value yes))
(not (answer (question tail-count)))
=>
(assert (answer (question tail-count) (value 1)))
(modify ?s (current-question Q1B1A))
(retract ?i)
(assert (user-input (valid no))))

(defrule Q1B1_valid_2
?s <- (state (current-question Q1B1))
?i <- (user-input (value 2) (valid yes))
(answer (question Q1) (value no))
(answer (question Q1B) (value yes))
(not (answer (question tail-count)))
=>
(assert (answer (question tail-count) (value 2)))
(modify ?s (current-question Q1B1B))
(retract ?i)
(assert (user-input (valid no))))

(defrule Q1B1_valid_more1
?s <- (state (current-question Q1B1))
?i <- (user-input (value >1) (valid yes))
(answer (question Q1) (value no))
(answer (question Q1B) (value yes))

```

```

(not (answer (question tail-count)))
=>
(assert (answer (question tail-count) (value >1)))
(modify ?s (current-question Q1B1B))
(retract ?i)
(assert (user-input (valid no))))

(defrule Q1B1_valid_integer_more
?s <- (state (current-question Q1B1))
?i <- (user-input (value ?v) (valid yes))
(answer (question Q1) (value no))
(answer (question Q1B) (value yes))
(not (answer (question tail-count)))
(test (integerp ?v))
(test (>= ?v 3))
=>
(assert (answer (question tail-count) (value >1)))
(modify ?s (current-question Q1B1B))
(retract ?i)
(assert (user-input (valid no))))

(defrule Q1B1_invalid
?i <- (user-input (value ?v) (valid yes))
(state (current-question Q1B1))
(test (not (or (eq ?v 1) (eq ?v 2) (eq ?v >1) (and (integerp ?v) (>= ?v 3)))))
=>
(printout t "Please answer 1, 2, or >1." crlf)
(retract ?i)
(assert (user-input (valid no))))

;; ===== Q1B1A: FEATHERY TAIL? =====
(defrule Q1B1A_prompt
(state (current-question Q1B1A))
(answer (question Q1) (value no))
(answer (question Q1B) (value yes))
(answer (question tail-count) (value 1))
(not (answer (question feathery)))
(user-input (valid no))
=>
(printout t "4. Does it have a feathery tail? (yes/no): ")
(bind ?response (lowercase (read)))
(assert (user-input (value ?response) (valid yes))))

(defrule Q1B1A_valid_yes
?s <- (state (current-question Q1B1A))
?i <- (user-input (value yes) (valid yes))
(answer (question Q1) (value no))
(answer (question Q1B) (value yes))
(answer (question tail-count) (value 1))
(not (answer (question feathery)))
=>
(assert (answer (question feathery) (value yes)))
(modify ?s (current-question conclusion))
(retract ?i)
(assert (user-input (valid no))))

(defrule Q1B1A_valid_no
?s <- (state (current-question Q1B1A))
?i <- (user-input (value no) (valid yes))
(answer (question Q1) (value no))
(answer (question Q1B) (value yes))
(answer (question tail-count) (value 1))
(not (answer (question feathery)))
=>
(assert (answer (question feathery) (value no)))
(modify ?s (current-question conclusion))
(retract ?i)
(assert (user-input (valid no))))

(defrule Q1B1A_invalid
?i <- (user-input (value ?v&:(not (or (eq ?v yes) (eq ?v no)))) (valid yes))
(state (current-question Q1B1A))
=>
(printout t "Please answer yes or no." crlf)
(retract ?i)
(assert (user-input (valid no))))

;; ===== Q1B1B: EXACTLY 2 TAILS? / or >1 path =====
(defrule Q1B1B_prompt_2
(state (current-question Q1B1B))

```



```

(answer (question Q1) (value no))
(answer (question Q1B) (value yes))
(answer (question tail-count) (value 2))
(not (answer (question exactly-2)))
(user-input (valid no))
=>
(printout t "4. Exactly 2 tails? (yes/no): ")
(bind ?response (lowercase (read)))
(assert (user-input (value ?response) (valid yes))))

(defrule Q1B1B_valid_2_yes
?s <- (state (current-question Q1B1B))
?i <- (user-input (value yes) (valid yes))
(answer (question Q1) (value no))
(answer (question Q1B) (value yes))
(answer (question tail-count) (value 2))
(not (answer (question exactly-2)))
=>
(assert (answer (question exactly-2) (value yes)))
(modify ?s (current-question conclusion))
(retract ?i)
(assert (user-input (valid no))))

(defrule Q1B1B_valid_2_no
?s <- (state (current-question Q1B1B))
?i <- (user-input (value no) (valid yes))
(answer (question Q1) (value no))
(answer (question Q1B) (value yes))
(answer (question tail-count) (value 2))
(not (answer (question exactly-2)))
=>
(assert (answer (question exactly-2) (value no)))
(modify ?s (current-question conclusion))
(retract ?i)
(assert (user-input (valid no))))

(defrule Q1B1B_prompt_more
(state (current-question Q1B1B))
(answer (question Q1) (value no))
(answer (question Q1B) (value yes))
(answer (question tail-count) (value >1))
(not (answer (question tails-longer)))
(user-input (valid no))
=>
(printout t "4. Are tails longer than body width? (yes/no): ")
(bind ?response (lowercase (read)))
(assert (user-input (value ?response) (valid yes))))

(defrule Q1B1B_valid_more_yes
?s <- (state (current-question Q1B1B))
?i <- (user-input (value yes) (valid yes))
(answer (question Q1) (value no))
(answer (question Q1B) (value yes))
(answer (question tail-count) (value >1))
(not (answer (question tails-longer)))
=>
(assert (answer (question tails-longer) (value yes)))
(modify ?s (current-question Q1B1B2))
(retract ?i)
(assert (user-input (valid no))))

(defrule Q1B1B_valid_more_no
?s <- (state (current-question Q1B1B))
?i <- (user-input (value no) (valid yes))
(answer (question Q1) (value no))
(answer (question Q1B) (value yes))
(answer (question tail-count) (value >1))
(not (answer (question tails-longer)))
=>
(assert (answer (question tails-longer) (value no)))
(modify ?s (current-question conclusion))
(retract ?i)
(assert (user-input (valid no))))

;; ===== Q1B1B2: THIN AND HAIRY TAILS? =====
(defrule Q1B1B2_prompt
(state (current-question Q1B1B2))
(answer (question Q1) (value no))
(answer (question Q1B) (value yes))
(answer (question tail-count) (value >1))

```

```

(answer (question tails-longer) (value yes))
(not (answer (question thin-hairy)))
(user-input (valid no))
=>
(printout t "5. Are the tails thin and hairy? (yes/no): ")
(bind ?response (lowercase (read)))
(assert (user-input (value ?response) (valid yes))))

(defrule Q1B1B2_valid_yes
?s <- (state (current-question Q1B1B2))
?i <- (user-input (value yes) (valid yes))
(answer (question Q1) (value no))
(answer (question Q1B) (value yes))
(answer (question tail-count) (value >1))
(answer (question tails-longer) (value yes))
(not (answer (question thin-hairy)))
=>
(assert (answer (question thin-hairy) (value yes)))
(modify ?s (current-question conclusion))
(retract ?i)
(assert (user-input (valid no))))

(defrule Q1B1B2_valid_no
?s <- (state (current-question Q1B1B2))
?i <- (user-input (value no) (valid yes))
(answer (question Q1) (value no))
(answer (question Q1B) (value yes))
(answer (question tail-count) (value >1))
(answer (question tails-longer) (value yes))
(not (answer (question thin-hairy)))
=>
(assert (answer (question thin-hairy) (value no)))
(modify ?s (current-question conclusion))
(retract ?i)
(assert (user-input (valid no))))

;; ===== Q1B2: SEGMENTED ABDOMEN? =====
(defrule Q1B2_prompt
(state (current-question Q1B2))
(answer (question Q1) (value no))
(answer (question Q1B) (value no))
(answer (question tail-count) (value 0))
(not (answer (question segmented)))
(user-input (valid no))
=>
(printout t "3. Does it have a segmented abdomen? (yes/no): ")
(bind ?response (lowercase (read)))
(assert (user-input (value ?response) (valid yes))))

(defrule Q1B2_valid_yes
?s <- (state (current-question Q1B2))
?i <- (user-input (value yes) (valid yes))
(answer (question Q1) (value no))
(answer (question Q1B) (value no))
(answer (question tail-count) (value 0))
(not (answer (question segmented)))
=>
(assert (answer (question segmented) (value yes)))
(modify ?s (current-question Q1B2A))
(retract ?i)
(assert (user-input (valid no))))

(defrule Q1B2_valid_no
?s <- (state (current-question Q1B2))
?i <- (user-input (value no) (valid yes))
(answer (question Q1) (value no))
(answer (question Q1B) (value no))
(answer (question tail-count) (value 0))
(not (answer (question segmented)))
=>
(assert (answer (question segmented) (value no)))
(modify ?s (current-question Q1B2B))
(retract ?i)
(assert (user-input (valid no))))

;; ===== Q1B2A: STUBBY WINGS? =====
(defrule Q1B2A_prompt
(state (current-question Q1B2A))
(answer (question Q1) (value no))
(answer (question Q1B) (value no))

```

```

(answer (question tail-count) (value 0))
(answer (question segmented) (value yes))
(not (answer (question stubby)))
(user-input (valid no))
=>
(printout t "4. Does it have stubby wings? (yes/no): ")
(bind ?response (lowercase (read)))
(assert (user-input (value ?response) (valid yes))))

(defrule Q1B2A_valid_yes
?s <- (state (current-question Q1B2A))
?i <- (user-input (value yes) (valid yes))
(answer (question Q1) (value no))
(answer (question Q1B) (value no))
(answer (question tail-count) (value 0))
(answer (question segmented) (value yes))
(not (answer (question stubby)))
=>
(assert (answer (question stubby) (value yes)))
(modify ?s (current-question conclusion))
(retract ?i)
(assert (user-input (valid no))))

(defrule Q1B2A_valid_no
?s <- (state (current-question Q1B2A))
?i <- (user-input (value no) (valid yes))
(answer (question Q1) (value no))
(answer (question Q1B) (value no))
(answer (question tail-count) (value 0))
(answer (question segmented) (value yes))
(not (answer (question stubby)))
=>
(assert (answer (question stubby) (value no)))
(modify ?s (current-question conclusion))
(retract ?i)
(assert (user-input (valid no))))

;; ===== Q1B2B: WINGS OVERLAP? =====
(defrule Q1B2B_prompt
(state (current-question Q1B2B))
(answer (question Q1) (value no))
(answer (question Q1B) (value no))
(answer (question tail-count) (value 0))
(answer (question segmented) (value no))
(not (answer (question wings-overlap)))
(user-input (valid no))
=>
(printout t "4. Do the wings overlap diagonally? (yes/no): ")
(bind ?response (lowercase (read)))
(assert (user-input (value ?response) (valid yes))))

(defrule Q1B2B_valid_yes
?s <- (state (current-question Q1B2B))
?i <- (user-input (value yes) (valid yes))
(answer (question Q1) (value no))
(answer (question Q1B) (value no))
(answer (question tail-count) (value 0))
(answer (question segmented) (value no))
(not (answer (question wings-overlap)))
=>
(assert (answer (question wings-overlap) (value yes)))
(modify ?s (current-question conclusion))
(retract ?i)
(assert (user-input (valid no))))

(defrule Q1B2B_valid_no
?s <- (state (current-question Q1B2B))
?i <- (user-input (value no) (valid yes))
(answer (question Q1) (value no))
(answer (question Q1B) (value no))
(answer (question tail-count) (value 0))
(answer (question segmented) (value no))
(not (answer (question wings-overlap)))
=>
(assert (answer (question wings-overlap) (value no)))
(modify ?s (current-question conclusion))
(retract ?i)
(assert (user-input (valid no))))

;; ===== CONCLUSION RULES =====

```

```

(defrule conclusion_water_spider
  (state (current-question conclusion))
  (answer (question Q1) (value yes))
  (answer (question Q1A) (value yes))
  =>
  (printout t crlf "==> Identification: Water Spider or Mite" crlf)
  (printout t "Why: 8 legs (4 pairs)" crlf crlf)
  (halt))

(defrule conclusion_crayfish
  (state (current-question conclusion))
  (answer (question Q1) (value yes))
  (answer (question Q1A) (value no))
  (answer (question Q1A2) (value yes))
  =>
  (printout t crlf "==> Identification: Freshwater Crayfish" crlf)
  (printout t "Why: big claws" crlf crlf)
  (halt))

(defrule conclusion_shrimp
  (state (current-question conclusion))
  (answer (question Q1) (value yes))
  (answer (question Q1A) (value no))
  (answer (question Q1A2) (value no))
  =>
  (printout t crlf "==> Identification: Freshwater Shrimp or Hog louse" crlf)
  (printout t "Why: no big claws" crlf crlf)
  (halt))

(defrule conclusion_alderfly
  (state (current-question conclusion))
  (answer (question Q1) (value no))
  (answer (question Q1B) (value yes))
  (answer (question tail-count) (value 1))
  (answer (question feathery) (value yes))
  =>
  (printout t crlf "==> Identification: Alderfly Larva" crlf)
  (printout t "Why: single feathery tail" crlf crlf)
  (halt))

(defrule conclusion_water_stick
  (state (current-question conclusion))
  (answer (question Q1) (value no))
  (answer (question Q1B) (value yes))
  (answer (question tail-count) (value 1))
  (answer (question feathery) (value no))
  =>
  (printout t crlf "==> Identification: Water Stick Insect or Water Scorpion" crlf)
  (printout t "Why: single tail, not feathery" crlf crlf)
  (halt))

(defrule conclusion_stonefly
  (state (current-question conclusion))
  (answer (question Q1) (value no))
  (answer (question Q1B) (value yes))
  (answer (question tail-count) (value 2))
  (answer (question exactly-2) (value yes))
  =>
  (printout t crlf "==> Identification: Stonefly Nymph" crlf)
  (printout t "Why: exactly 2 tails" crlf crlf)
  (halt))

(defrule conclusion_mayfly
  (state (current-question conclusion))
  (answer (question Q1) (value no))
  (answer (question Q1B) (value yes))
  (answer (question tail-count) (value >1))
  (answer (question tails-longer) (value yes))
  (answer (question thin-hairy) (value yes))
  =>
  (printout t crlf "==> Identification: Mayfly Nymph" crlf)
  (printout t "Why: multiple thin hairy tails longer than body" crlf crlf)
  (halt))

(defrule conclusion_damselfly
  (state (current-question conclusion))
  (answer (question Q1) (value no))
  (answer (question Q1B) (value yes))
  (answer (question tail-count) (value >1))
  (answer (question tails-longer) (value yes))

```

```

(answer (question thin-hairy) (value no))
=>
(printout t crlf "==> Identification: Damselfly Nymph" crlf)
(printout t "Why: multiple tails longer than body, not thin/hairy" crlf crlf)
(halt))

(defrule conclusion_dragonfly_more
  (state (current-question conclusion))
  (answer (question Q1) (value no))
  (answer (question Q1B) (value yes))
  (answer (question tail-count) (value >1))
  (answer (question tails-longer) (value no))
  =>
  (printout t crlf "==> Identification: Dragonfly Nymph" crlf)
  (printout t "Why: multiple tails not longer than body" crlf crlf)
  (halt))

(defrule conclusion_water_boatman
  (state (current-question conclusion))
  (answer (question Q1) (value no))
  (answer (question Q1B) (value no))
  (answer (question tail-count) (value 0))
  (answer (question segmented) (value no))
  (answer (question wings-overlap) (value yes))
  =>
  (printout t crlf "==> Identification: Water Boatman" crlf)
  (printout t "Why: wings overlap diagonally" crlf crlf)
  (halt))

(defrule conclusion_water_beetle
  (state (current-question conclusion))
  (answer (question Q1) (value no))
  (answer (question Q1B) (value no))
  (answer (question tail-count) (value 0))
  (answer (question segmented) (value no))
  (answer (question wings-overlap) (value no))
  =>
  (printout t crlf "==> Identification: Water Beetle" crlf)
  (printout t "Why: no segmented abdomen, wings don't overlap diagonally" crlf crlf)
  (halt))

(defrule conclusion_dragonfly
  (state (current-question conclusion))
  (answer (question Q1) (value no))
  (answer (question Q1B) (value no))
  (answer (question tail-count) (value 0))
  (answer (question segmented) (value yes))
  (answer (question stubby) (value yes))
  =>
  (printout t crlf "==> Identification: Dragonfly Nymph" crlf)
  (printout t "Why: segmented abdomen, stubby wings" crlf crlf)
  (halt))

(defrule conclusion_caddis
  (state (current-question conclusion))
  (answer (question Q1) (value no))
  (answer (question Q1B) (value no))
  (answer (question tail-count) (value 0))
  (answer (question segmented) (value yes))
  (answer (question stubby) (value no))
  =>
  (printout t crlf "==> Identification: Caseless Caddis Fly or Beetle Larvae" crlf)
  (printout t "Why: segmented abdomen, no stubby wings" crlf crlf)
  (halt))

(defrule fallback
  (state (current-question conclusion))
  =>
  (printout t crlf "Unable to identify. Please check your answers." crlf)

  (halt))

```

Appendix B (Panopto link)

Video Link

Appendix C (Freshwater Arthropod Identification)

Freshwater Arthropod Identification (2mm+ excludes skaters and other surface animals)



Figure 8: Original identification tree